
◆ 1. File-based Routing

Summary:

Next.js uses a **file-system-based router** — the folder structure in `/pages` (or `/app`) directly maps to routes.

Example:

```
pages/  
├─ index.js      → /  
├─ about.js     → /about  
└─ blog/[slug].js → /blog/:slug (dynamic)
```

Interview Q&A:

Q1: How is routing different from React Router? A1: Next.js doesn't need a routing config file — it's automatic based on folder structure.

Q2: How do you define a dynamic route? A2: Use square brackets:

```
/blog/[slug].js → /blog/hello-world
```

Q3: Can you have nested routes? A3:  Yes — just nest folders:

```
pages/dashboard/settings.js → /dashboard/settings
```

Q4: Where do 404 pages go? A4: Create `pages/404.js` — Next.js uses this automatically for unknown routes.

◆ 2. SSR vs CSR

Summary:

Server-Side Rendering (SSR) and Client-Side Rendering (CSR) are two different approaches for when and where HTML is generated.

Comparison:

Type	When rendered	SEO Friendly?	Initial Load
SSR	At request-time on server	 Yes	Slower (but fresh)
CSR	In browser (JS loads data)	 No (initially)	Faster (but blank initially)

Interview Q&A:

Q1: When should you use SSR in Next.js? A1: For **SEO-sensitive, frequently updated** pages like blogs, eCommerce product pages.

Q2: How do you implement SSR in Next.js? A2:

```
export async function getServerSideProps(context) {  
  return { props: { data: ... } }  
}
```

Q3: When is CSR better? A3: For **dashboards, auth-required pages**, or pages behind login (where SEO doesn't matter).

Q4: Can you mix SSR and CSR in the same app? A4:  Yes — each page can independently choose how it fetches data.

◆ 3. API Routes

Summary:

Next.js lets you create **backend APIs** inside the app using `pages/api/*.js` (or `app/api/route.js` in app directory).

Example:

```
// pages/api/hello.js
export default function handler(req, res) {
  res.status(200).json({ message: 'Hello API' })
}
```

Interview Q&A:

Q1: Why does Next.js support API routes? A1: To let you build **fullstack apps** — backend logic lives alongside frontend.

Q2: Can API routes access environment variables? A2:  Yes — use `process.env.MY_SECRET`.

Q3: How do you restrict methods (GET/POST)? A3:

```
if (req.method !== 'POST') {
  res.status(405).end()
}
```

Q4: Are API routes serverless? A4:  In Vercel, they run as serverless functions. In custom hosting, they're handled like any Node.js backend.

◆ 4. Image Optimization

Summary:

Next.js has a built-in `<Image />` component that **automatically optimizes** images for **size, format**, and **lazy loading**.

Example:

```
import Image from 'next/image'

<Image
  src="/banner.jpg"
  width={800}
  height={600}
  alt="Banner"
/>
```

Interview Q&A:

Q1: Why use `<Image />` instead of `<img>` ? A1:

- Automatic resizing
 - WebP support
 - Lazy loading
 - Responsive images
-

Q2: What are the drawbacks? A2:

- Works only with statically hosted images (unless `remotePatterns` configured)
 - Requires specifying width/height or using `fill`
-

Q3: How to allow external images? A3: Add to `next.config.js` :

```
images: {
  remotePatterns: [
    {
      protocol: 'https',
```

```
    hostname: 'cdn.example.com',  
  },  
],  
}
```

Q4: Is image optimization server-side or client-side? A4: Handled by the **Next.js** server (Node.js or Vercel serverless).

◆ 5. Dynamic Routing

Summary:

Dynamic routes allow you to create pages where part of the URL is a variable (e.g., `/blog/[slug]`).

Example:

```
// pages/blog/[slug].js  
  
export async function getStaticProps({ params }) {  
  const post = await getPost(params.slug)  
  return { props: { post } }  
}  
  
export async function getStaticPaths() {  
  return {  
    paths: [{ params: { slug: 'hello-world' } }],  
    fallback: false,  
  }  
}
```

Interview Q&A:

Q1: How do you define a dynamic route? A1: Use `[param]` in file name like `pages/post/[id].js` .

Q2: What does `getStaticPaths()` do? A2: Tells Next.js which paths to pre-render at **build time** for dynamic pages.

Q3: What's the role of `fallback` ? A3:

- `false` : unknown paths = 404
 - `true` : render on first request
 - `'blocking'` : SSR during build
-

Q4: Can you use dynamic routing in the `/app` directory? A4:  Yes — files are named `[slug]/page.js` .

◆ 6. Middleware

Summary:

Middleware runs **before a request is completed**, allowing you to intercept, rewrite, redirect, or block requests.

Example: `/middleware.ts`

```
import { NextResponse } from 'next/server'

export function middleware(req) {
  if (!req.cookies.auth) {
    return NextResponse.redirect(new URL('/login', req.url))
  }
}
```

Interview Q&A:

Q1: Where does middleware run in the stack? A1: Between the incoming request and the rendering layer — before page logic or static asset handling.

Q2: Can middleware run on specific paths? A2:  Yes, using the `matcher` in `middleware.ts`.

```
export const config = {  
  matcher: ['/dashboard/:path*'],  
}
```

Q3: Can you use middleware for geolocation or A/B tests? A3:  Yes — especially on Vercel, where location and headers are available server-side.

Q4: Can middleware return JSX? A4:  No — middleware can only redirect, rewrite, or return a response (not render components).

◆ 7. Static Generation (SSG)

Summary:

Next.js can pre-render pages to **static HTML at build time**, improving speed and reducing server load.

Example:

```
export async function getStaticProps() {  
  return { props: { data: "Pre-rendered!" } }  
}
```

Interview Q&A:

Q1: When should you use SSG? A1: For content that doesn't change often, like marketing pages, blogs, or docs.

Q2: Can SSG support dynamic routes? A2:  Yes — with `getStaticPaths()` to generate paths at build time.

Q3: What if you need to update content later? A3: Use ISR (Incremental Static Regeneration) or trigger a **rebuild** on deploy.

Q4: What's the fallback to SSG? A4: Fallback enables rendering new pages **after deployment**, useful for large or growing content sites.

◆ 8. Routing Config

Summary:

You can configure **custom routing behaviors** using `next.config.js` — such as rewrites, redirects, and headers.

Example:

```
module.exports = {
  async redirects() {
    return [
      {
        source: '/old',
        destination: '/new',
        permanent: true,
      },
    ],
  },
}
```

Interview Q&A:

Q1: Difference between `redirects` and `rewrites` ? A1:

- `redirect` : URL changes in browser (302 or 301)

- `rewrite` : URL stays same, content comes from new path

Q2: Can you add headers like CORS or CSP here? A2:  Yes — using `headers()` config block.

Q3: When are routing configs applied? A3: At build time and served at edge in Vercel.

Q4: Can you make locale-based routing? A4:  Yes — with `i18n` config in `next.config.js`.

◆ 9. App Dir Structure (`/app`)

Summary:

The new **App Router** in Next.js 13+ replaces the traditional `pages/` directory with `app/`, enabling **React Server Components**, nested layouts, and full file-based convention.

Example:

```
app/
├── layout.js      // shared layout
├── page.js        // route: /
└── dashboard/
    ├── page.js    // route: /dashboard
    └── layout.js   // nested layout
```

Interview Q&A:

Q1: How is `app/` different from `pages/` ? A1:

- No `getServerSideProps`, `getStaticProps`
- Uses `page.js`, `layout.js`, and `loading.js`
- Allows **streaming** and **server components**

Q2: Can `app/` and `pages/` coexist? A2:  Yes, but only one handles a route — `app/` takes priority if a route exists in both.

Q3: Where do you define metadata or head tags in `app/` ? A3: In `layout.js` using the `metadata` object or `<Head />` in `client` components.

Q4: Can `app/` be used with dynamic routing? A4:  Yes — use `[slug]/page.js` .

◆ 10. Server Actions (Next 13+)

Summary:

Server Actions let you run **mutating logic** (e.g., DB updates) directly on the server, triggered from UI — no need for separate API routes.

Example:

```
'use server'

export async function saveForm(formData) {
  await db.save(formData)
}
```

Then use it in a form:

```
<form action={saveForm}>
  <input name="email" />
  <button type="submit">Submit</button>
</form>
```

Interview Q&A:

Q1: What problem do Server Actions solve? A1: Removes the need for `/api/*` routes or client-side `fetch()` — executes logic server-side on form submission or action call.

Q2: Are Server Actions secure? A2:  Yes — they only run on the server and can't be accessed directly via URL.

Q3: Can you use them with props or hooks? A3:  No — they must be called from form `action`, not within `useEffect`.

Q4: Do Server Actions support revalidation or redirection? A4:  Yes — they integrate with `revalidatePath()`, `redirect()`, etc.

◆ 11. Middleware Caching

Summary:

Middleware in Next.js runs at the **edge**, and Vercel **automatically caches** middleware results based on URL, headers, cookies, etc.

Concepts:

- Cached unless `next: { revalidate: 0 }` is set
 - Can manually control cache with headers in `NextResponse`
-

Interview Q&A:

Q1: What types of caching are possible in Next.js? A1:

- Static caching (ISR)
- Middleware response caching
- Image and asset caching
- Fetch caching (Next 13)

Q2: How to disable caching for a middleware route? A2: Add headers:

```
res.headers.set('Cache-Control', 'no-store')
```

Q3: Are middleware results cached across deploys? A3: ❌ No — all middleware responses are reset on redeploy.

Q4: What tool manages edge caching in Vercel? A4: Vercel Edge Network — it supports CDN-level cache control tied to middleware.

◆ 12. Image Components

🔍 Summary:

Next.js offers an advanced `<Image />` component that supports:

- Auto format (`webp` , `avif`)
 - Lazy loading
 - Responsive resizing
 - CDN delivery with `next/image`
-

✅ Example:

```
<Image
  src="https://cdn.example.com/image.png"
  width={600}
  height={400}
  alt="Optimized"
/>
```

💡 Interview Q&A:

Q1: Why is `<Image />` preferred over `<img />` ? A1:

- Improves performance
 - Reduces layout shift (with known sizes)
 - Lazy-loads automatically
 - Supports responsive sizes via `sizes`
-

Q2: What does `fill` prop do? A2: Makes image fill its parent container, useful with relative layout styles.

Q3: Can you use remote images with `<Image />` ? A3:  Yes — configure domains in `next.config.js` .

Q4: Is the `<Image />` component supported in `app/` directory? A4:  Yes — but needs `use client` if used with interactive logic.

◆ 13. Routing API

Summary:

Next.js exposes an internal **Routing API** via hooks like `useRouter()` (pages dir) or `usePathname()` , `useSearchParams()` (app dir) to manage and react to navigation.

Example:

```
// pages/_app.js or a client component
import { useRouter } from 'next/router'

const router = useRouter()
router.push('/about')
```

Interview Q&A:

Q1: Can you programmatically navigate in Next.js? A1:  Yes, using:

- `useRouter().push('/route')` in `pages/`
 - `useRouter()` or `usePathname()` in `app/` (client components only)
-

Q2: How do you handle route change events? A2:

```
router.events.on('routeChangeStart', () => console.log('Changing'))
```

Q3: How to get the current route? A3:

- In `pages`: `router.pathname`
 - In `app` dir: `usePathname()` hook
-

Q4: Is `next/link` required for navigation? A4:  Recommended — it prefetches routes for faster UX.

◆ 14. API Middlewares

Summary:

In Next.js API routes (`pages/api/*.js`), you can use **middleware-like functions** for auth, logging, body parsing, etc.

Example:

```
import nc from 'next-connect' // middleware runner
const handler = nc()
  .use(authMiddleware)
  .get((req, res) => res.send('ok'))

export default handler
```

Interview Q&A:

Q1: Does Next.js API routing support Express middleware? **A1:** Not directly, but with `next-connect` or `iron-session` you can add middleware patterns.

Q2: Can you parse JSON body manually in API routes? **A2:**  Yes, but by default it's already parsed (`req.body`) — unless you set `bodyParser: false`.

Q3: Can you apply middleware globally to all API routes? **A3:** Not natively — you'd need to wrap each route or use custom server (e.g., Express with Next).

15. Fallback Pages

Summary:

Fallback behavior determines how **dynamic static pages** (`getStaticPaths`) handle unknown paths at runtime.

Fallback Modes in `getStaticPaths()` :

```
export async function getStaticPaths() {
  return {
    paths: [{ params: { slug: 'post1' } }],
    fallback: 'blocking' // or true / false
  }
}
```

Interview Q&A:

Q1: What does `fallback: false` do? **A1:** Only generate listed paths; unknown routes show 404.

Q2: What's the difference between `true` and `'blocking'` ? A2:

- `true` : renders skeleton (loading state), then fetches content
 - `'blocking'` : waits for full data before rendering
-

Q3: Can fallback pages use `ISR` after generation? A3:  Yes — fallback pages are cached and re-used via ISR settings.

◆ 16. ISR (Incremental Static Regeneration)

Summary:

ISR allows statically generated pages to be **updated in the background, without full redeploy** — the perfect mix of static + dynamic.

Example:

```
export async function getStaticProps() {
  return {
    props: { data: "Hello" },
    revalidate: 60 // seconds
  }
}
```

Interview Q&A:

Q1: How does ISR improve performance? A1: You get fast, cached static content that **self-updates** after a set interval — no rebuild needed.

Q2: What happens when a request comes in during revalidation? A2: The cached page is served while Next.js **rebuilds** it in the background.

Q3: Can you trigger ISR manually? A3:  Yes — using [On-Demand Revalidation](#)

```
res.revalidate('/path')
```

◆ 17. Preview Mode

Summary:

Preview Mode lets you see **draft content** from a headless CMS or unpublished data without rebuilding the site.

Example:

```
// pages/api/preview.js
export default function handler(req, res) {
  res.setPreviewData({})
  res.writeHead(307, { Location: '/' })
  res.end()
}
```

Interview Q&A:

Q1: When should you use Preview Mode? A1: When working with CMS-driven content — to preview unpublished changes in a live site.

Q2: How do you check if preview mode is active in `getStaticProps` ? A2:

```
export async function getStaticProps({ preview }) {
  if (preview) {
    // fetch draft data
  }
}
```

Q3: How to disable preview mode? A3: Use:

```
res.clearPreviewData()
```

◆ 18. Middleware Order

Summary:

If multiple middlewares (rewrites, redirects, etc.) are applied, their order **matters**. Execution flows top-down based on route matching.

Control via:

```
export const config = {  
  matcher: ['/admin/:path*', '/user/:path*'],  
}
```

Interview Q&A:

Q1: In what order are multiple middleware matchers evaluated? A1: Top to bottom — earlier matches take precedence.

Q2: Can you conditionally bypass middleware? A2:  Yes — use logic in middleware to `return` `NextResponse.next()` early.

Q3: Can you chain multiple middleware files? A3:  No native chaining — write your logic as a single `middleware.ts` file.

◆ 19. Server Context

Summary:

In the App Router (`/app`), **server context** lets you share things like cookies, headers, user info across layouts, pages, and components.

Example:

```
// In a server component
import { cookies } from 'next/headers'

export default function Dashboard() {
  const token = cookies().get('auth_token')
}
```

Interview Q&A:

Q1: What are `headers()` and `cookies()` functions in Next.js 13+? A1: Built-in APIs to access request metadata in server components.

Q2: Can you use these in `client` components? A2:  No — only in **server components** (or use `fetch()` to call an API that accesses them).

Q3: What's the benefit of Server Context over passing props? A3: Centralized, request-based access — avoids prop drilling, supports auth & personalization cleanly.

20. SEO Optimizations

Summary:

Next.js includes built-in ways to optimize SEO via `Head` , `metadata` , dynamic `<title>` , OG tags, sitemaps, etc.

✓ Example:

```
export const metadata = {  
  title: 'My Page',  
  description: 'SEO-friendly description',  
}
```

Or in pages:

```
import Head from 'next/head'  
  
<Head>  
  <title>Custom Page</title>  
  <meta name="description" content="SEO stuff" />  
</Head>
```

💬 Interview Q&A:

Q1: How does SSR help SEO? A1: It allows crawlers to receive a **fully rendered HTML page**, improving indexability and metadata parsing.

Q2: How to add OG (Open Graph) tags? A2:

```
<meta property="og:title" content="Page Title" />  
<meta property="og:image" content="/img.png" />
```

Q3: How to generate a sitemap in Next.js? A3: Use libraries like `next-sitemap` or generate it manually in `pages/api/sitemap.xml`.

🎁 Bonus 4 Next.js Topics

◆ 1. On-Demand ISR (Incremental Static Regeneration)

Summary:

You don't have to wait for the revalidation window — you can **manually trigger a rebuild** for any page via API.

Example: `/pages/api/revalidate.js`

```
export default async function handler(req, res) {
  const secret = req.query.secret
  if (secret !== process.env.REVALIDATE_SECRET) {
    return res.status(401).json({ message: 'Invalid token' })
  }
  await res.revalidate('/blog/post-slug')
  return res.json({ revalidated: true })
}
```

Interview Q&A:

Q: When do you use on-demand ISR? A: When content editors update a CMS and need instant page refresh without redeploy.

2. Custom `_document.js` vs `_app.js`

Summary:

These files allow you to control how **pages are rendered**:

- `_document.js` : for changing HTML template (runs on server only)
 - `_app.js` : wraps every page (runs on both server + client)
-

Example:

```
// pages/_document.js
<Html lang="en">
  <Head />
```

```
<body>
  <Main />
  <NextScript />
</body>
</Html>
```

Interview Q&A:

Q: When should you use `_document.js` ? A: For setting global HTML-level tags like `lang`, structured data, fonts — **server only**.

Q: What's `_app.js` good for? A: Add global CSS, layout wrappers, context providers — **every page** passes through it.

◆ 3. Bundle Analyzer & Build Optimization

Summary:

You can analyze your build size to detect bloated packages and lazy load only when needed.

Setup:

```
npm install @next/bundle-analyzer
```

```
// next.config.js
const withBundleAnalyzer = require('@next/bundle-analyzer')({
  enabled: process.env.ANALYZE === 'true',
})
module.exports = withBundleAnalyzer({})
```

Interview Q&A:

Q: How do you reduce Next.js bundle size? A:

- Lazy load components
 - Remove unused dependencies
 - Dynamically import heavy libraries (`import('lib')`)
 - Use modern images
-
-

◆ 4. Middleware + Authentication Guards

🔍 Summary:

You can create **global route protection** by checking auth cookies/tokens in middleware.

✅ Example:

```
import { NextResponse } from 'next/server'

export function middleware(req) {
  const token = req.cookies.get('auth_token')
  if (!token) {
    return NextResponse.redirect(new URL('/login', req.url))
  }
}
```

💬 Interview Q&A:

Q: What are benefits of doing auth in middleware? A: It's **edge-run**, fast, and avoids loading full page logic if user is unauthenticated.
